

算竞

STL核心容器

vector (动态数组)

```
#include <vector>
vector<int> v;
v.push_back(1); // 尾部插入元素
v.pop_back(); // 删除尾部元素
v.size(); // 返回元素个数
v.empty(); // 判断是否为空
v[i]; // 随机访问第i个元素 (不检查边界)
v.at(i); // 随机访问第i个元素 (检查边界, 越界抛出异常)
v.front(); // 访问首元素
v.back(); // 访问尾元素
v.clear(); // 清空所有元素
v.begin(), v.end(); // 返回首尾迭代器, 常用于遍历
```

- **特点:** 支持快速随机访问, 尾部插入删除效率高 (摊还 $O(1)$), 中间插入删除慢 ($O(n)$)。

deque (双端队列)

```
#include <deque>
deque<int> dq;
dq.push_front(1); // 头部插入
dq.push_back(2); // 尾部插入
dq.pop_front(); // 头部删除
dq.pop_back(); // 尾部删除
dq[i]; // 随机访问
dq.size(); // 元素个数
```

- **特点:** 首尾操作均为 $O(1)$, 支持随机访问, 比 vector 更适合频繁首尾插入的场景。

set/multiset (有序集合)

```
#include <set>
set<int> s;
s.insert(3); // 插入元素
s.erase(3); // 删除值为3的元素
s.find(3); // 查找元素, 返回迭代器
s.count(3); // 统计元素出现次数 (0或1)
s.lower_bound(3); // 第一个  $\geq 3$  的位置
```

```
s.upper_bound(3);    // 第一个 >3 的位置
s.size();           // 元素个数
```

- **特点：**基于红黑树实现，自动排序，`set` 不允许重复，`multiset` 允许重复，插入查找删除均为 $O(\log n)$ 。

map/multimap (有序映射)

```
#include <map>
map<string, int> m;
m["key"] = 10;        // 插入或修改键值对
m.erase("key");       // 删除键为"key"的元素
m.find("key");         // 查找键，返回迭代器
m.count("key");        // 判断键是否存在 (0或1)
m.size();              // 键值对个数
```

- **特点：**按键自动排序，`map` 键唯一，`multimap` 键可重复，操作复杂度 $O(\log n)$ 。

unordered_set/unordered_map (哈希容器)

```
#include <unordered_set>
#include <unordered_map>
unordered_set<int> us;
us.insert(5);          // 插入
us.count(5);           // 查找 (返回0或1)

unordered_map<string, int> um;
um["key"] = 100;       // 插入/修改
```

- **特点：**基于哈希表实现，平均操作复杂度 $O(1)$ ，无序，适合对性能要求高的场景。

stack (栈)

```
#include <stack>
stack<int> st;
st.push(1);            // 入栈
st.pop();              // 出栈 (无返回值)
st.top();              // 获取栈顶元素
st.empty();            // 判断是否为空
st.size();             // 栈中元素个数
```

- **特点：**后进先出 (LIFO)，所有操作 $O(1)$ 。

queue (队列)

```
#include <queue>
queue<int> q;
q.push(1);           // 入队
q.pop();             // 出队 (无返回值)
q.front();           // 队首元素
q.back();            // 队尾元素
q.empty();           // 判断空
```

- **特点：**先进先出 (FIFO) ，操作 $O(1)$ 。

priority_queue (优先队列)

```
#include <queue>
priority_queue<int> pq; // 默认大根堆
pq.push(3);             // 插入
pq.pop();               // 删除堆顶
pq.top();               // 获取堆顶元素
```

- **特点：**基于堆实现，默认大顶堆，插入删除 $O(\log n)$ ，适合求最大/最小值。

常用算法函数

排序与查找

```
#include <algorithm>
sort(v.begin(), v.end());           // 升序排序
sort(v.begin(), v.end(), greater<int>()); // 降序排序
binary_search(v.begin(), v.end(), x); // 二分查找 (需有序)
lower_bound(v.begin(), v.end(), x);   // 第一个  $\geq x$  的位置
upper_bound(v.begin(), v.end(), x);   // 第一个  $> x$  的位置
```

元素操作

```
reverse(v.begin(), v.end()); // 反转容器
unique(v.begin(), v.end());   // 去重相邻重复元素 (常与erase连用)
merge(v1.begin(), v1.end(), v2.begin(), v2.end(), result.begin()); // 合并两个有序容器
fill(v.begin(), v.end(), val); // 填充指定值
```

最大最小

```
max(a, b);           // 返回较大值
min(a, b);           // 返回较小值
max_element(v.begin(), v.end()); // 返回最大元素迭代器
min_element(v.begin(), v.end()); // 返回最小元素迭代器
```

数学函数

基础运算

```
#include <cmath>
abs(x);           // 绝对值 (整数/浮点数)
sqrt(x);          // 平方根 (浮点数)
pow(x, y);        // x的y次方 (浮点数)
floor(x);         // 向下取整
ceil(x);          // 向上取整
round(x);         // 四舍五入
```

数论函数

```
// 最大公约数 (递归实现)
int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}

// 最小公倍数
int lcm(int a, int b) {
    return a / gcd(a, b) * b; // 防止溢出: a * b / gcd(a, b)
}

// 快速幂 (取模版)
long long quick_pow(long long a, long long b, long long mod) {
    long long res = 1;
    while (b) {
        if (b & 1) res = (res * a) % mod;
        a = (a * a) % mod;
        b >>= 1;
    }
    return res;
}

// 素数判断 (试除法)
bool is_prime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}
```

```

}

// 筛法求素数 (埃氏筛)
void sieve(int n) {
    vector<bool> is_prime(n + 1, true);
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; i * i <= n; i++) {
        if (is_prime[i]) {
            for (int j = i * i; j <= n; j += i) {
                is_prime[j] = false;
            }
        }
    }
}

```

排列组合

```

// 阶乘
long long factorial(int n) {
    long long res = 1;
    for (int i = 1; i <= n; i++) res *= i;
    return res;
}

// 组合数 C(n, m)
long long combination(int n, int m) {
    if (m > n) return 0;
    long long num = 1, den = 1;
    for (int i = 1; i <= m; i++) {
        num *= (n - i + 1);
        den *= i;
    }
    return num / den;
}

```

字符串处理

字符串分割

```

#include <sstream>
#include <vector>
vector<string> split(const string& s, char delim) {
    vector<string> res;
    stringstream ss(s);
    string item;
    while (getline(ss, item, delim)) {
        res.push_back(item);
    }
}

```

```
    return res;
}
// 示例: split("a,b,c", ',') -> {"a", "b", "c"}
```

大小写转换

```
#include <algorithm>
#include <cctype>
string s = "AbC";
// 转小写
transform(s.begin(), s.end(), s.begin(), ::tolower);
// 转大写
transform(s.begin(), s.end(), s.begin(), ::toupper);
```

字符串转数字

```
#include <string>
string str = "123";
int num = stoi(str);    // string to int
long lng = stol(str);   // string to long
double db = stod(str);  // string to double
```

数论函数

```
// 最大公约数
int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}

// 最小公倍数
int lcm(int a, int b) {
    return a / gcd(a, b) * b;
}

// 快速幂 (取模)
long long qpow(long long a, long long n, long long mod) {
    long long ans = 1;
    while (n) {
        if (n & 1) ans = ans * a % mod;
        a = a * a % mod;
        n >>= 1;
    }
    return ans;
}
```

```
// 素数判断
bool is_prime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) return false;
    return true;
}
```

排列组合

```
// 阶乘
long long factorial(int n) {
    long long res = 1;
    for (int i = 1; i <= n; i++) res *= i;
    return res;
}

// 组合数 C(n, m)
long long comb(int n, int m) {
    if (m > n) return 0;
    long long num = 1, den = 1;
    for (int i = 1; i <= m; i++) {
        num *= (n - i + 1);
        den *= i;
    }
    return num / den;
}
```

字符串处理

分割字符串

```
#include <sstream>
vector<string> split(const string& s, char delim) {
    vector<string> res;
    stringstream ss(s);
    string item;
    while (getline(ss, item, delim)) res.push_back(item);
    return res;
}
```

大小写转换

```
#include <algorithm>
transform(s.begin(), s.end(), s.begin(), ::tolower); // 转小写
```

```
transform(s.begin(), s.end(), s.begin(), ::toupper); // 转大写
```

输入输出优化

```
ios::sync_with_stdio(false);  
cin.tie(nullptr);
```

算法模板

快速排序

```
void quick_sort(int a[], int l, int r) {  
    if (l >= r) return;  
    int i = l, j = r, pivot = a[l];  
    while (i < j) {  
        while (i < j && a[j] >= pivot) j--;  
        while (i < j && a[i] <= pivot) i++;  
        if (i < j) swap(a[i], a[j]);  
    }  
    swap(a[l], a[i]);  
    quick_sort(a, l, i-1);  
    quick_sort(a, i+1, r);  
}
```

归并排序

```
void merge_sort(int a[], int l, int r, int temp[]) {  
    if (l >= r) return;  
    int mid = (l + r) / 2;  
    merge_sort(a, l, mid, temp);  
    merge_sort(a, mid+1, r, temp);  
    int i = l, j = mid+1, k = l;  
    while (i <= mid && j <= r) {  
        if (a[i] <= a[j]) temp[k++] = a[i++];  
        else temp[k++] = a[j++];  
    }  
    while (i <= mid) temp[k++] = a[i++];  
    while (j <= r) temp[k++] = a[j++];  
    for (int idx = l; idx <= r; idx++) a[idx] = temp[idx];  
}
```

1. 排序与查找


```
#include <algorithm>
sort(v.begin(), v.end());           // 排序
binary_search(v.begin(), v.end(), x); // 二分查找
lower_bound(v.begin(), v.end(), x);  // 第一个≥x的位置
upper_bound(v.begin(), v.end(), x);  // 第一个>x的位置
```

2. 元素操作

```
reverse(v.begin(), v.end());        // 反转
unique(v.begin(), v.end());          // 去重 (相邻)
merge(v1.begin(), v1.end(), v2.begin(), v2.end(), v.begin()); // 合并
```

3. 数值操作

```
#include <numeric>
iota(v.begin(), v.end(), 1);        // 填充连续值
partial_sum(v.begin(), v.end(), v.begin()); // 部分和
```

Dijkstra最短路

```
#include <queue>
vector<pair<int, int>> g[N]; // {目标, 权重}
int dist[N];

void dijkstra(int st, int n) {
    fill(dist, dist + n, INT_MAX);
    priority_queue<pair<int, int>> pq; // {-距离, 节点}
    dist[st] = 0;
    pq.push({0, st});

    while (!pq.empty()) {
        int d = -pq.top().first, u = pq.top().second;
        pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : g[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({-dist[v], v});
            }
        }
    }
}
```

并查集

```
int parent[N], rank[N];

void init(int n) {
    for (int i = 0; i < n; i++) {
        parent[i] = i;
        rank[i] = 0;
    }
}

int find(int x) {
    return parent[x] == x ? x : parent[x] = find(parent[x]);
}

void unite(int x, int y) {
    x = find(x), y = find(y);
    if (x == y) return;
    if (rank[x] < rank[y]) swap(x, y);
    parent[y] = x;
    if (rank[x] == rank[y]) rank[x]++;
}
```